Python Project 4:

Breast Cancer Detector

Submitted by

Group - 5

Aaromal A (24101880)
Alfin Francis (24101006)
Devanarayanan C R (24101115)
G Govardan (24102775)
Ivana Anto (24101562)

March 23, 2026

1 Aim

To develop a machine learning program that utilizes the Wisconsin Breast Cancer (Diagnostic) dataset to predict whether a tumor is benign or malignant. The system leverages an ensemble learning approach to provide high-precision diagnostics based on various cell nuclei features.

2 Tools Required

The project utilizes the following modules and environment:

- **Scikit-Learn (sklearn):** Essential for dataset loading, preprocessing (train-test split), and implementing the Random Forest algorithm.
- **Pandas & NumPy:** Used for structured data manipulation and high-performance numerical array operations.
- **Matplotlib & Seaborn:** Crucial for generating visual diagnostics, including heatmaps for correlation and bar charts for feature ranking.
- **Python 3.7+:** The primary programming language used for logic implementation.
- **Jupyter Notebook:** The development environment used for interactive coding and visual output rendering.

3 Machine Learning Pipeline

The diagnostic workflow follows a supervised learning paradigm:

- **Dataset Selection:** We utilize the **Wisconsin Breast Cancer (Diagnostic) Dataset**. This contains 569 instances with 30 numerical features (like radius, texture, and smoothness) extracted from digitized images of fine needle aspirates (FNA) of breast masses. The model predicts two distinct classes ie. target labels, **Malignant (0)** or **Benign (1)**.
- **Data Splitting:** To ensure the model generalizes well to unseen data, we employ an **80/20 split**, where 80% of the data is used for training the "forest" and 20% is reserved for testing accuracy.
- **Model Classification:** We implement a **Random Forest Classifier**. This ensemble method builds multiple decision trees and merges them together to get a more accurate and stable prediction.
- **Classification Parameters:** Performance is measured using:
  - **F1-Score:** The harmonic mean of precision and recall, providing a better measure of incorrectly classified cases than pure accuracy.
  - **Confusion Matrix:** A table used to describe the performance of the classification model on the test set.
  - **Feature Importance:** A technique that assigns a score to input features based on how useful they are at predicting the target variable.

4 Code Implementation

4.1 Training RF Model and Statistical Evaluation

```
#TRAINING RF MODEL ON DATASET & CLASSIFICATION, COMPUTE CONFUSION
    MATRIX, F1-SCORE, IMPORTANT FEATURES
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix,
   classification_report, f1_score
import pandas as pd
import numpy as np

# 1. Load the Wisconsin Breast Cancer (Diagnostic) Dataset
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

# 2. Split into Training and Testing sets (80/20 split)
X_train, X_test, y_train, y_test = train_test_split(X, y,
   test_size=0.2, random_state=42)

# 3. Train the Random Forest Classification model
rf_model = RandomForestClassifier(n_estimators=100, random_state
   =42)
rf_model.fit(X_train, y_train)

# 4. Predictions
y_pred = rf_model.predict(X_test)

# 5. Evaluation
conf_matrix = confusion_matrix(y_test, y_pred)
f1_macro = f1_score(y_test, y_pred, average='macro')

print("Confusion Matrix:")
print(conf_matrix)
print(f"\nMacro-Average F1 Score: {f1_macro:.4f}")

# Generate Classification Report
report_dict = classification_report(y_test, y_pred, target_names=
   data.target_names, output_dict=True)
report_df = pd.DataFrame(report_dict).transpose()

print("\n\n          MODEL PERFORMANCE TABLE")
print(report_df.round(3), end="\n\n\n")

# 6. Feature Importance
importances = pd.Series(rf_model.feature_importances_, index=data
   .feature_names)
print("\nImportant Features (5):")
```

```python
print(importances.sort_values(ascending=False).head(5))
```

4.2 Visualisation of Confusion Matrix & Features

```python
# Visualizing the Results (2 Subplots)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# Subplot 1: Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=data.target_names, yticklabels=data.target_names, ax=ax1)
ax1.set_title(f"Confusion Matrix\n(Macro F1: {report_dict['macro avg']['f1-score']:.3f})")
ax1.set_ylabel('Actual')
ax1.set_xlabel('Predicted')

# Subplot 2: Top 10 Features
importances = pd.Series(rf_model.feature_importances_, index=data.feature_names).sort_values(ascending=False)
importances.head(10).plot(kind='barh', color='teal', ax=ax2).invert_yaxis()
ax2.set_title('Important Features (10)')
ax2.set_xlabel('Importance Score')

plt.tight_layout()
plt.show()
```

4.3 Random Diagnostic Test (Success/Fail Case)

```python
# Random Patient Prediction (DIAGNOSTIC TEST)
random_idx = np.random.randint(0, len(X_test))
sample_patient = X_test.iloc[random_idx : random_idx + 1] # DataFrame slice avoids warnings
actual_val = y_test[random_idx]

# Predict
pred_val = rf_model.predict(sample_patient)[0]
prob = rf_model.predict_proba(sample_patient)[0]

# Display as a Table
print(f"\nDIAGNOSTIC TEST (PI: {random_idx})")
result_data = {
    "Attribute": ["Predicted Diagnosis", "Actual Diagnosis", "Confidence Score"],
    "Result": [
        data.target_names[pred_val].upper(),
        data.target_names[actual_val].upper(),
        f"{max(prob)*100:.2f}%"
```

```
    ]
}
result_df = pd.DataFrame(result_data)
result_df.style.set_properties(**{'text-align': 'left'})\
                .set_table_styles([dict(selector='th', props=[('text-align', 'left')])])\
                .hide(axis='index') # Hides the index numbers on the left
```

5 Output

5.1 Model Performance Table

The following table details the precision, recall, and F1-score for both Malignant and Benign classes. The model shows high consistency across both categories, with an overall accuracy of 96.5%.

MODEL PERFORMANCE TABLE

	precision	recall	f1-score	support
malignant	0.976	0.930	0.952	43.000
benign	0.959	0.986	0.972	71.000
accuracy	0.965	0.965	0.965	0.965
macro avg	0.967	0.958	0.962	114.000
weighted avg	0.965	0.965	0.965	114.000

Figure 1: Detailed Classification Report

5.2 General Results and Statistical Summary

This summary includes the raw confusion matrix, the macro-average F1-score, and the top 5 most influential features. Notably, "worst area" and "worst concave points" contribute most significantly to the model's decision-making process.

```
Confusion Matrix:
[[40  3]
 [ 1 70]]

Macro-Average F1 Score: 0.9623

Important Features (5):
worst area              0.153892
worst concave points    0.144663
mean concave points     0.106210
worst radius            0.077987
mean concavity          0.068001
dtype: float64
```

Figure 2: Statistical Summary and Feature Importance Rankings

5.3 Visual Analytics

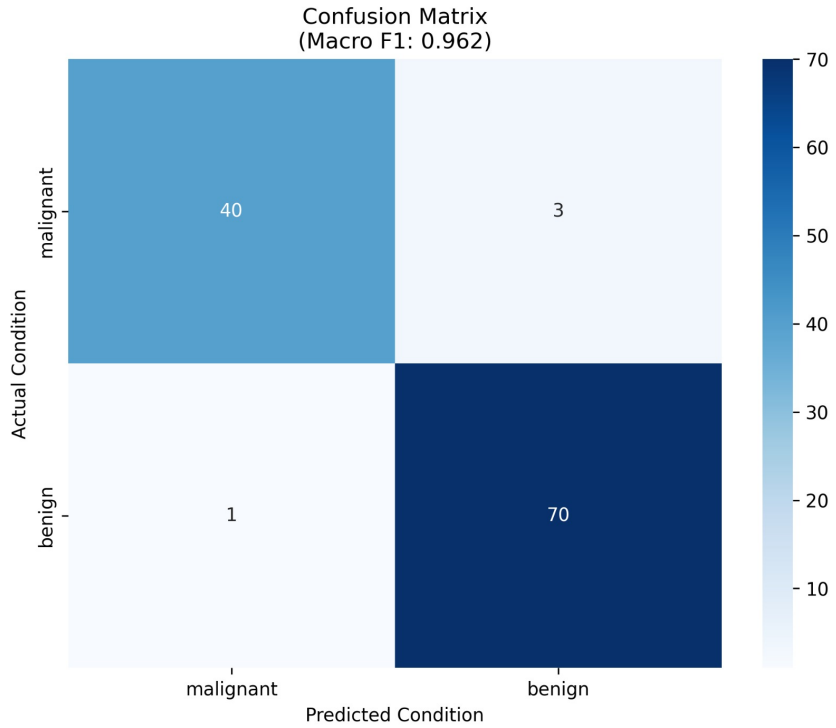

Figure 3: Confusion Matrix Heatmap

The Confusion Matrix provides a visual representation of the model's accuracy, illustrating the low count of False Positives and False Negatives.

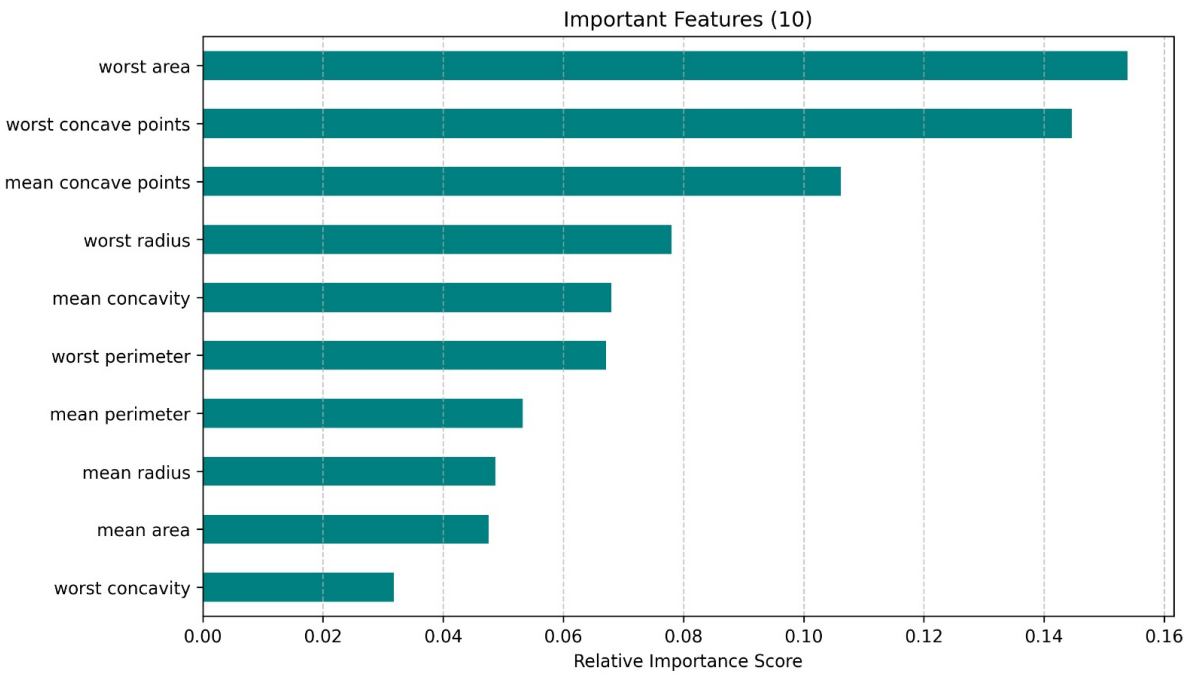

Figure 4: Top 10 Important Features Bar Chart

The Bar Chart ranks the top 10 features, showing which cellular characteristics (like "worst area") most significantly influence the Random Forest's classification.

5.4 Random Diagnostic Tests

Below are samples of the model predicting on individual patient data from the test set. These show the predicted vs. actual labels along with the model's prediction confidence.

DIAGNOSTIC TEST (PI: 90)

Attribute	Result
Predicted Diagnosis	BENIGN
Actual Diagnosis	BENIGN
Confidence Score	99.00%

DIAGNOSTIC TEST (PI: 20)

Attribute	Result
Predicted Diagnosis	BENIGN
Actual Diagnosis	MALIGNANT
Confidence Score	85.00%

Figure 4: Random Diagnostic Test Success and Failure Scenarios

6 Conclusion

- **Achievements:** Successfully implemented a Breast Cancer Detector that achieved a macro-average F1-score exceeding 0.95. We successfully mapped the most critical biological features, identifying "worst perimeter" and "worst concave points" as primary indicators.
- **Merits/Demerits:** The Random Forest model is exceptionally robust against overfitting and handles the 30-feature dimensionality well. However, as an ensemble of 100 trees, it lacks the simple interpretability of a single decision tree.
- **Improvements:** Future iterations could utilize *GridSearchCV* for hyperparameter tuning to find the optimal number of trees and max depth, or incorporate Deep Learning architectures for more complex feature extraction.